

Team Jailhouse Lock

Josue Peraza Navarrete, Yash Patel, Derrick Casseus, Emmanuel Martinez, Joseph Falco, Leo Oliveira

PASSWORD MANAGER APP



Architecture and Design Specifications

Table of Contents

1 - Introduction

- 1.1 - Purpose & Objectives
- 1.2 - Scope of the Architecture
- 1.3 - Key Design Principles
- 1.4 - Terminology and Definitions
- 1.5 - References

2 - System Overview

- 2.1 - High-Level Description
- 2.2 - Technology Stack Overview
- 2.3 - Box-and-Line System Diagram

3 - Architectural Models

- 3.1 - Model-View-Controller (MVC)
- 3.2 - Repository Model
- 3.3 - Client-Server Architecture

4 - Design Patterns Implemented

- 4.1 - Object Pool Pattern
- 4.2 - Decorator Design Pattern
- 4.3 - Observer Design Pattern

5 - Conclusion

1. Introduction

1.1 Purpose & Objectives

The GateKeep Architecture and Design Documentation serves as a detailed framework delineating the internal architecture, communication protocols, and system interactions that collectively yield a secure, cross-platform desktop password management solution. This document provides an in-depth exploration of the system's construction by going beyond mere functionality to assess the technical choices, frameworks, and architectural paradigms that direct its development.

GateKeep aims to revolutionize password security through superior engineering and an exceptional user experience. This document elucidates the design patterns employed, the stratification of systems, component responsibilities, and the underlying design philosophy that underpins this vault-grade software.

1.2 Scope of the Architecture

This specification elaborates on the comprehensive full-stack architecture and design of GateKeep, detailing its Electron-based user interface alongside its C++ backend services. It encompasses visual modeling techniques, including component and sequence diagrams, as well as a modular breakdown of the system. Additionally, it presents implementation strategies that adhere to the established best practices and architectural standards outlined in the course syllabus.

The design choices prioritize scalability, maintainability, modularity, and, fundamentally, a security-first approach, ranging from the integration of cryptographic protocols to the management of UI event handling.

1.3 Key Design Principles

GateKeep's architecture adheres to the following core principles:

- **Separation of Concerns** - Via MVC and Repository layers
- **Secure-by-Design** - Through AES-256, Argon2, and platform-native credential APIs
- **Cross-Platform Consistency** - With Electron and native C++ bindings
- **Reusability & Scalability** - Using a modular pattern-based implementation
- **Real-Time Responsiveness** - Through Observer-driven state updates

1.4 Terminology and Definitions

Term	Definition
Box and Line Graph	Visual modeling technique to represent system architecture
MVC	Architectural pattern for decoupling components
Repository	Centralized layer for data access management
JS	Programming language used mainly for web development
Electron JS	Framework for creating cross-platform desktop applications using web technologies
Node-Gyp	Build tool for compiling native add-ons in Node.js
AES-256	Advanced Encryption Standard using 256-bit keys
Argon2	Cryptographic algorithm for hashing passwords
PostgreSQL	Relational database system used for secure data storage
Object Pool	Pattern for reusing objects to manage resources efficiently

1.5 References

- [Electron Documentation](#)
- [Node.js and Node-Gyp Build Process](#)
- [PostgreSQL Official Documentation](#)
- [Argon2 Specification – IETF](#)
- [AES Cryptographic Standard – NIST](#)
- [*Design Patterns: Elements of Reusable Object-Oriented Software*, Gamma et al.](#)

2. System Overview

2.1 High-Level Description

GateKeep is a modern and secure password management system designed for individuals and organizations that need an efficient, reliable, and user-friendly way to manage credentials. Our users can use the application across multiple devices without having to worry about data synchronization. While the frontend UI offers a smooth experience, our backend provides high performance and data security using multiple encryption systems. The application allows users to store passwords and access them without any friction. Customers will also be able to see features such as password suggestions and strength checking. Users will be notified if their credentials are weak or being reused. By keeping a minimalistic and clean user interface, we make it easy for users to navigate through the application.

2.2 Technology Stack Overview

Frontend

Desktop application/Browser Extension: This is where all the user input takes place.

Users are able to send requests and receive data.

Programming Language: Primarily used C++ and JavaScript for app development.

Electron Framework: This framework allows seamless development of cross-platform applications.

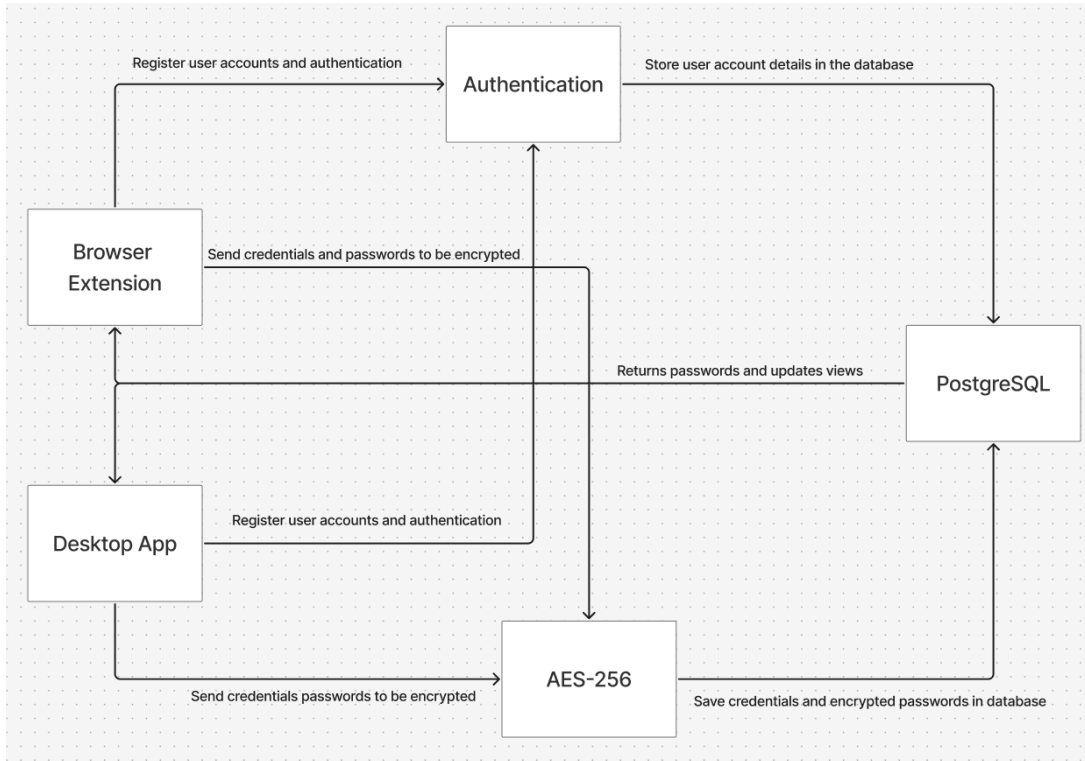
Backend

PostgreSQL: A PostgreSQL database used to store data securely. When the user requests to view passwords, PostgreSQL fulfills the request by sending the decrypted version of the passwords.

API: Calls are made to Clearbit API for organizations' logos to be displayed along with the passwords.

2.3 Box and Line System Diagram

The structure of our application can be understood using a box-and-line diagram. This view helps visualize the flow of data into different components of the application. The Desktop App is the main component of the software, along with the Browser Extension being an extended feature of the software. Both handle user interaction through a user graphical interface. The GUI interacts with the Authentication system to handle user account registration and secure login. The account information gets stored in a PostgreSQL database where the data is hashed. Whenever the user creates or adds a new password, it gets encrypted using an AES-256 algorithm to then be stored in the PostgreSQL database. Upon verification of the user, the decrypted passwords get sent back to the Desktop App or Browser Extension for the user to use.

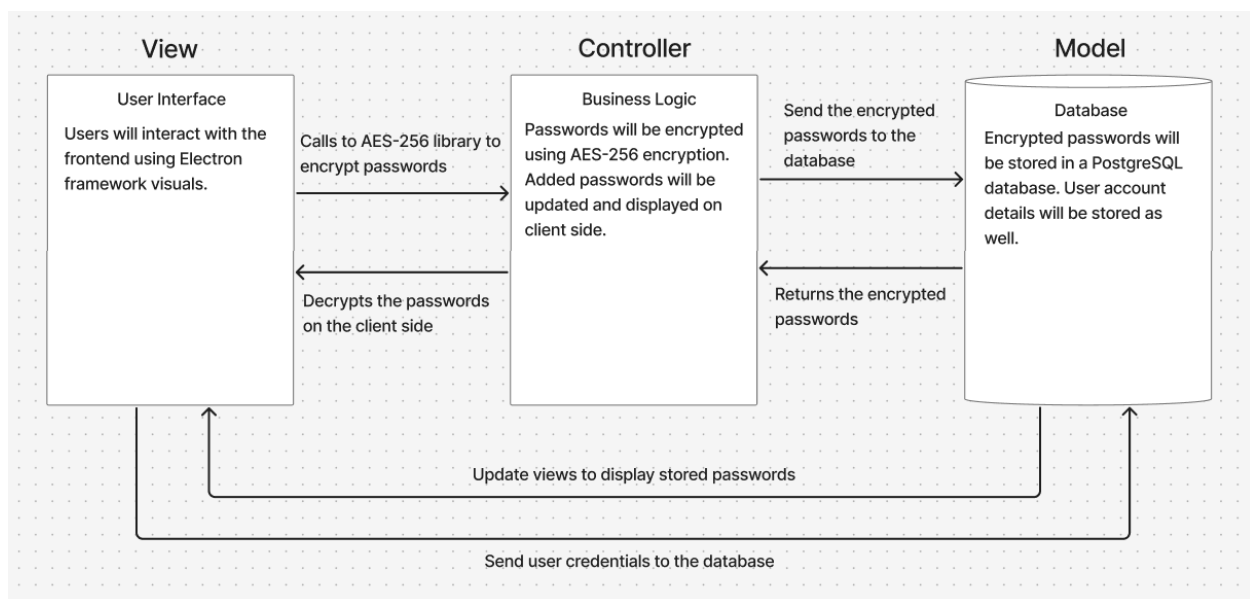


3. Architecture Models

3.1 Model-View-Controller (MVC)

GateKeep's desktop application and web browser extension can be simplified to a Model-View-Controller architecture pattern. The MVC architecture pattern separates an application into three interconnected components, making it easier to see what is responsible for specific aspects of the application's functionality. Our View is the interface that users will interact with. The View has communication between the Model and the Controller. Databases typically represent the Model component of MVC models. The Model component of GateKeep is our database PostgreSQL, which is used as our database management system to store user

login credentials and encrypted passwords. PostgreSQL is crucial for the functionality of GateKeep and for the security of our users. The Model retrieves and stores encrypted passwords from the Controller, which will then be sent to the View for the user to see. The Controller is the business logic of GateKeep, which is the AES-256 library used to encrypt passwords. The Controller receives plaintext passwords from the View, encrypts them, and sends them to the database. When encrypted passwords are sent back to the Controller, it will decrypt them and send them to the View to be displayed to the user.



3.2 Repository Model

The development process behind GateKeep has made heavy use of the repository model. Jailhouse Lock, the team behind GateKeep, has been utilizing GitHub, one of the primary examples of a repository, to develop this app through committing new files or changes to keep GateKeep up to date. Through the usage of GitHub, JailHouse Lock has been able to ensure that all of GateKeep's essential files are kept together in one place to allow for easier interactions. Additionally, GitHub makes it so that the files within its repository are only able to be changed by those with specific permissions who already made the edits outside of GitHub itself.

However, GitHub also faces the same flaws as many other examples of the repository model in the sense that if the website gets compromised, it would heavily impact the app development process. However, JailHouse Lock ensures these risks won't be exploited by keeping their GitHub as a private repository alongside having additional copies of this GitHub in Visual Studio Code to make sure development can still progress in the case of GitHub being compromised.

The screenshot displays a GitHub repository interface for 'PasswordManager_Jeffreys'. The repository is private and has 2 branches and 0 tags. The main branch is 'main'. The repository description states: 'Made for the use of establishing a cybersecurity password system. Meant only to be seen by Hofstra students.' The repository has 57 commits, 0 stars, 2 watchers, and 0 forks.

The file list includes:

- Backend: code clean up (3 days ago)
- Documents: Create Meeting Highlights (2 months ago)
- Extension: Progress on Chrome Extension (3 days ago)
- Frontend: Updated version with div (2 days ago)
- build: code clean up (3 days ago)
- .gitignore: getting started with electron ui (2 months ago)
- README.md: Update README.md (2 months ago)
- alerts.js: Weak passwords list updated (2 weeks ago)
- binding.gyp: updated code to support windows (last week)
- package-lock.json: hashing password to gatekeep (3 weeks ago)
- package.json: Reapply "Working for windows OS" (5 days ago)

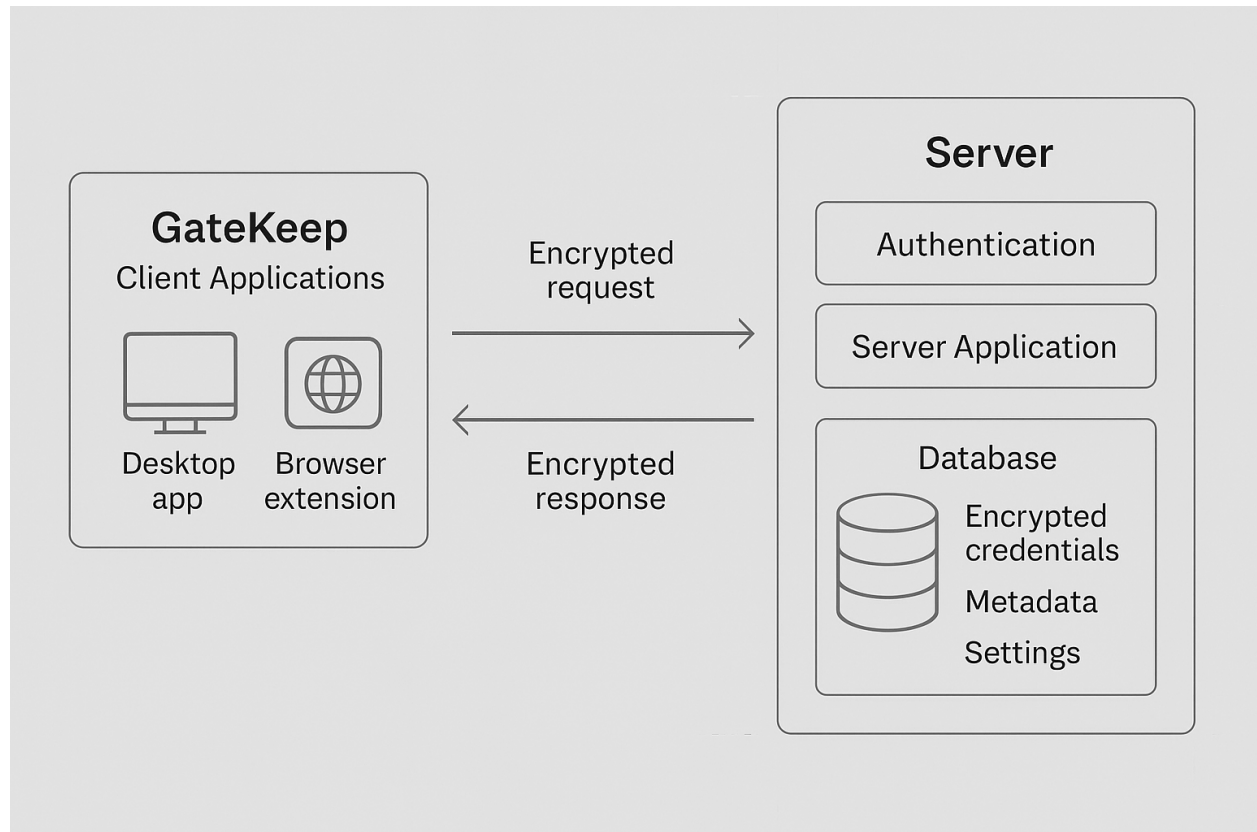
The README section is titled 'PasswordSystem_Jeffreys' and contains the text: 'Made for the use of establishing a cybersecurity password system. Meant only to be seen by Hofstra students.' Below the title, it lists 'Rule 1:' followed by the instruction: 'Comment on your posts with your what changes you've made alongside potential issues you've came across.'

The sidebar on the right shows repository statistics and language usage:

- Releases:** No releases published. [Create a new release](#)
- Packages:** No packages published. [Publish your first package](#)
- Contributors:** 5 contributors
- Languages:**
 - Makefile: 30.3%
 - Python: 29.9%
 - C++: 13.7%
 - HTML: 9.9%
 - JavaScript: 9.7%
 - CSS: 6.4%
 - C: 0.1%
- Suggested workflows:** Based on your tech stack

3.3 Client-Server Architecture

GateKeep employs a client-server architecture to ensure secure and scalable interactions. This model supports cross-platform access, modularity, and strong security. Certain components, such as Client Applications, provide a cross-platform interface for desktop and browser extensions to handle user interactions, encryption, and secure communication across different devices. Another is the Server application, which manages requests, enforces security, and interacts with the database, which itself stores encrypted credentials, metadata, and settings using the repository model to keep the integrity of the security of all the user data and communication across the board. On the topic of communication flow, the architecture allows for the client to send an encrypted request with authentication and actions to increase the security of the private information as well. The server then verifies, processes, and encrypts the response alongside decrypting and displaying the results as the user's password information, allowing for swift and ease of access. Finally, some design considerations to take into account for the architecture are the scalability to ensure that GateKeeps supports multiple users efficiently and modularity to update the new information that gets added, such as passwords and new websites that get added to the information that the user has. Finally, the last design consideration, security, is the most important priority, ensuring end-to-end encryption that protects the users, clients, and information in general to provide a safe and secure outlook for the application.



4. Design Patterns Implemented

4.1 Object Pool

The Object Pool design pattern allows for the quick accessibility and storage of reusable objects. GateKeep implements this pattern to store its users' credentials. This design pattern creates a system in which the application is asked for an object in its pool or database. If the object exists, it is called for use by the client. If the object does not exist, it is added to the pool. GateKeep's implementation of the object pool design involves the calling of a password based on its website and returning the password back into the pool when not in use. The purpose behind the

implementation of this pattern is to facilitate the quick acquisition and storage of credentials to boost performance.

4.2 Decorator Design Pattern

The decorator design pattern is the pattern of software continuing to grow more complex as time goes on through multiple new additions to the code. This design pattern has had heavy prominence throughout the development process of GateKeep. GateKeep has gone through multiple internal updates throughout its development cycle in order to ensure that requested features are fully implemented into the system. As an example, earlier builds of GateKeep only allowed the user to type in passwords for a specific website, whereas later builds of GateKeep built on this feature through the use of its database, properly storing those used passwords and keeping them secure until they need to be called on once again. The team behind GateKeep not only implemented a new feature into the software, they did so by building off of what already existed, a key characteristic of the decorator design pattern. This same pattern applied for every other function of GateKeep that the team initially designed and built upon later, proving to be a prominent design pattern throughout the entire development process.

4.3 Observer Design Pattern

The observer design pattern involves the connection between multiple existing functions in the code where the actions of one function causes changes to all of the observing functions. This is best seen with the process of adding a new password in GateKeep due to the multiple additional events that occur directly afterwards. In the case where a user enters a new password into the system, multiple background functions are called upon to update key aspects of the system related to password creation and storage. The database is given a hashed and encrypted version

of the credentials and stores them there until a user later deletes or calls for them. At the same time, the user interface is informed about the same actions and updates to show the user their specific password. An api is additionally updated with this information and pulls the logo associated with the website the password is for and gives it to the user interface to place next to the credentials. In one singular action, the multiple observers take the information surrounding the password getting added and run their own functions to adapt to those changes. The observer design pattern is essential for GateKeep to properly run and ensure their users' passwords are secure.

5. Conclusion

The architecture of GateKeep is a prime example of a methodical, security-focused engineering approach meant to provide a cross-platform, scalable password management system. As described in the document, the core models and design patterns that support the system's frontend and backend are Model-View-Controller (MVC), Repository, and Client-Server. These patterns guarantee safe communication across the platform, improve maintainability, and enable modularity.

Furthermore, GateKeep heavily utilizes multiple common design patterns to ensure it can appropriately perform whatever functions are requested of it. The object pool pattern greatly improves GateKeep's performance while allowing for cross-platform compatibility, native cryptographic processing capabilities, and secure data persistence. The decorator design pattern allows the development process behind GateKeep to swiftly proceed without major reworks needing to be done whenever an update occurs. The observer design pattern connects multiple important objects within GateKeep together through assigning observers to objects, allowing a

change in one object to only affect the objects that need to be aware of it. These patterns were essential in making GateKeep an optimal password storage system that can adapt to whatever is required of it.

GateKeep strictly complies with current security standards by using modern encryption methods like AES-256 and Argon2. It is this architecture that positions GateKeep as more than just a tool. GateKeep is a carefully designed digital vault that is suited to the demands of modern users, and as the system develops, it provides a strong basis for future improvements.